

SecurePR: Reusable Pull Request Security Harness and Security Gate

GitHub Repository

<https://github.com/poncema4/SecurePR>

Marco Ponce

Seton Hall University

M.S. Cybersecurity

ISCI 6201: Secure Software Development

December 2026

Abstract

SecurePR is a reusable pull request security harness designed to integrate multiple automated security controls into a single GitHub Actions security gate. Instead of replacing individual security tools, SecurePR coordinates existing tools and project tests, collects their results, evaluates a common policy, and reports a final PASS or BLOCK decision. The current implementation combines repository profiling, application tests, dependency auditing, Gitleaks secret detection, Semgrep static analysis, CodeQL analysis, SARIF result aggregation, and OWASP Top 10:2025 finding-level mapping.

The project was developed as a practical Secure Software Development and DevSecOps implementation. A major goal was to make security checks repeatable and understandable to developers while preserving evidence from the underlying tools. SecurePR also supports reusable GitHub Actions workflows so that the security harness can be consumed by other repositories rather than copied into every project.

The system was evaluated using a reviewed 37-case benchmark containing 15 true positives, 16 true negatives, 0 false positives, and 6 false negatives. This produced an accuracy of 83.78

1 Introduction

Modern software development increasingly incorporates security checks directly into the development lifecycle. Rather than waiting until an application is completed before looking for vulnerabilities, secure software development practices encourage organizations to identify and address security problems throughout development [5].

Continuous integration systems provide a practical location for these checks because they can automatically evaluate changes whenever developers submit or update pull requests.

The problem is that security checks are often distributed across several independent tools. A project may use one tool for secret detection, another for static analysis, another for dependency vulnerabilities, and another for source-code security analysis. While these tools are useful individually, their results can be difficult to combine into one consistent development decision.

SecurePR was developed to address this problem.

SecurePR is a reusable pull request security harness that sits above existing security tools. It does not attempt to become another replacement for Semgrep, CodeQL, Gitleaks, or dependency scanners. Instead, it provides the orchestration, policy, evidence, and reporting layer around them.

The central idea is:

Pull Request → Security Checks → Evidence → Policy → PASS/BLOCK

This report describes the architecture, implementation, security controls, OWASP mapping, controlled experiments, benchmark results, limitations, and future work associated with SecurePR.

2 Problem Statement

Security tools provide valuable information, but using several tools independently can create an inconsistent developer experience. Different scanners may use different output formats, finding identifiers, severity models, and reporting mechanisms.

This creates several practical problems:

- Security findings may be scattered across different tools.
- Developers may not have one clear PASS/BLOCK decision.
- Duplicate findings may make reports harder to understand.
- Security results may not be connected to a recognized security framework.
- Repositories may require significant workflow duplication.
- A scanner result does not automatically provide a clear remediation workflow.

SecurePR addresses the integration problem rather than attempting to solve every possible vulnerability-detection problem.

The project therefore focuses on building a consistent security gate that combines several existing controls and provides developers with a single result.

3 Project Objectives

The objectives of the SecurePR project were to:

- Build a reusable GitHub Actions security gate.
- Run automated software tests as part of the security process.
- Identify supported repository languages and dependency manifests.
- Perform dependency auditing.
- Detect hardcoded secrets and credentials.
- Run custom Semgrep security rules.
- Run CodeQL security analysis.
- Aggregate security results using SARIF.
- Produce a consistent PASS or BLOCK decision.
- Map applicable findings to OWASP Top 10:2025 categories.
- Preserve security evidence from workflow runs.
- Provide useful developer-facing reporting.
- Evaluate the implementation using a reviewed benchmark.
- Demonstrate the remediation lifecycle using controlled pull requests.
- Design the system so that the security workflow can be reused by other repositories.

4 Security Background

4.1 Secure Software Development

NIST’s Secure Software Development Framework (SSDF) provides practices that can be integrated into software development processes to reduce vulnerabilities and address their causes [5]. SecurePR does not attempt to implement the complete SSDF. Instead, it applies the general principle of integrating security verification into the development process.

The pull request is a useful enforcement point because it occurs before a proposed change is merged into the main development branch.

4.2 Static Analysis

Static Application Security Testing analyzes software without requiring the application to operate normally during the analysis. Static analysis can identify insecure programming patterns, dangerous functions, and other security conditions.

SecurePR uses two complementary analysis approaches.

Semgrep provides custom rules that target security patterns selected for the project. These rules include hardcoded credentials, privileged usernames, unsafe evaluation, and disabled TLS verification.

CodeQL provides an additional source-code analysis engine. GitHub describes CodeQL as a code analysis engine that represents code as data and uses queries to identify vulnerabilities and errors [3].

Using more than one analysis engine provides broader coverage than relying on one custom rule set alone.

4.3 Secret Detection

Hardcoded credentials are dangerous because source code can be copied, distributed, or retained in version-control history.

SecurePR uses Gitleaks for secret detection [4]. It also includes targeted Semgrep rules for high-confidence hardcoded credentials.

This creates two complementary detection mechanisms rather than relying on only one scanner.

4.4 Software Supply Chain Security

Modern software depends on third-party libraries, package managers, source repositories, and CI/CD systems. A vulnerable dependency can therefore introduce security risk even when the application’s own source code appears correct.

OWASP identifies dependency management, source control, build systems, and CI/CD infrastructure as important components of the software supply chain [7].

SecurePR therefore includes dependency auditing as part of its pull request security process.

4.5 OWASP Top 10:2025

The OWASP Top 10:2025 is an awareness framework for important web application security risks [6].

SecurePR uses the framework as a reporting and coverage mechanism. It does not claim that the automated checks completely validate every OWASP category.

The current mapping includes categories such as A04:2025 Cryptographic Failures, A05:2025 Injection, and A07:2025 Authentication Failures when applicable findings are identified.

5 System Architecture

SecurePR is implemented as a GitHub Actions-based security harness. The target repository contains the application being evaluated, while the SecurePR repository contains the security tooling and reusable workflow components.

The overall workflow can be summarized as:

**Developer → Pull Request → GitHub Actions → SecurePR → Security Controls →
Evidence → Policy → PASS/BLOCK**

GitHub Actions workflows are YAML-defined automated processes that execute jobs in response to repository events [1]. SecurePR uses pull request events to automatically reevaluate changes as developers update their branches.

5.1 SecurePR Architecture Diagram

The architecture of SecurePR is shown in Figure 1. The diagram is generated directly in $\text{\LaTeX}$ using TikZ rather than being imported as an external screenshot.

5.2 Repository Profiling

SecurePR includes a repository profiling script that identifies supported programming languages and common dependency manifests.

The current language profiles include:

- C/C++
- C#
- Go
- Java/Kotlin
- JavaScript/TypeScript
- Python
- Ruby
- Rust
- Swift

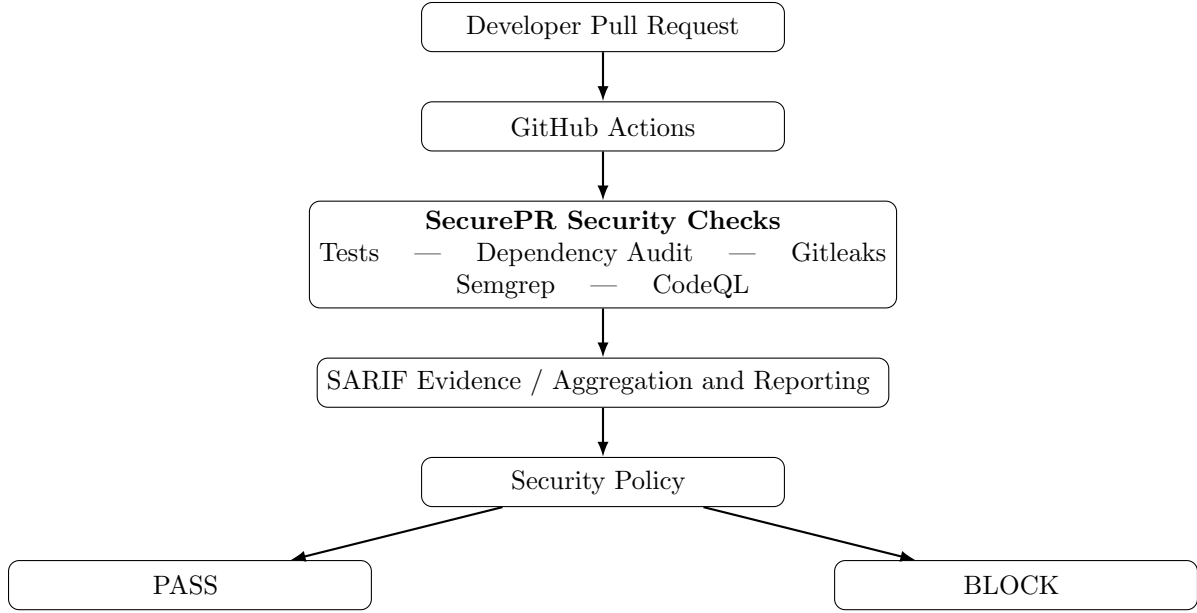


Figure 1: Simplified SecurePR architecture showing the pull request lifecycle, security controls, evidence aggregation, and final PASS/BLOCK policy decision.

Common dependency manifests are also identified, including Python, Node.js, Go, Rust, Java, and Ruby dependency files.

Unsupported languages are reported rather than silently treated as secure.

5.3 Workflow Structure

The primary workflow is named:

SecurePR Security Gate

The workflow performs the major stages shown in Table 1.

Stage	Purpose
Runtime and policy	Establish execution environment
Repository profile	Identify languages and manifests
Tests	Verify application behavior
Dependency audit	Identify dependency vulnerabilities
Gitleaks	Detect potential secrets
Semgrep	Run custom security rules
CodeQL	Perform source-code analysis
SARIF summary	Aggregate security evidence
Policy evaluation	Determine PASS/BLOCK
Reporting	Publish results to GitHub Actions

Table 1: Major SecurePR workflow stages.

6 Implementation

6.1 Security Checks

The workflow executes several independent security controls. Core security checks are not configured to ignore failures. If a required security control fails, the workflow can ultimately produce a BLOCK result.

The individual tools remain responsible for their specialized analysis. SecurePR is responsible for coordinating those results.

6.2 Custom Semgrep Rules

The custom Semgrep rules currently include:

- hardcoded passwords and credentials,
- Python hardcoded credential assignments,
- hardcoded privileged usernames,
- unsafe `eval()` or `exec()` usage,
- disabled TLS verification in Python.

One important implementation improvement occurred during testing. A benchmark case containing a Python assignment such as:

```
password = "demo_password"
```

was not reliably detected by the original generic rule. A Python AST-based rule was subsequently added to improve detection of this pattern.

This demonstrates an important property of the project: the security harness itself was tested and improved based on observed behavior.

6.3 SARIF Aggregation

Security tools can produce SARIF results that contain structured information about findings.

SecurePR includes a SARIF summarization script that groups findings using the artifact URI, starting line, rule identifier, and message.

The tool name is not used as the sole identity of a finding. This prevents the same underlying issue from being unnecessarily repeated simply because multiple security tools reported it.

6.4 PASS/BLOCK Policy

SecurePR intentionally uses only two final security outcomes:

- PASS
- BLOCK

A PASS means that the configured automated controls did not identify a condition that requires the pull request to be blocked.

A BLOCK means that at least one required control or policy condition failed.

A PASS does not mean that the repository has been proven secure. Human review is still recommended for both outcomes.

7 OWASP Top 10:2025 Mapping

SecurePR maps applicable automated security controls to the OWASP Top 10:2025 as a coverage framework [6].

The current custom rule mappings are shown in Table 2.

SecurePR Finding	OWASP Mapping
Hardcoded password	A04, A07
Python hardcoded credential	A04, A07
Privileged username	A01, A07
Unsafe eval/exec	A05
Unverified TLS	A02, A04

Table 2: Current SecurePR OWASP Top 10:2025 finding mappings.

The mapping is intentionally finding-level.

For example, a hardcoded password causes A04 and A07 to be reported as blocked because those categories are explicitly mapped to the finding. It does not cause every OWASP category to fail.

This distinction corrected an earlier implementation issue in which a Semgrep failure could cause multiple unrelated OWASP categories to appear blocked. The corrected implementation only blocks categories for which an actual mapped finding exists.

A PASS for an OWASP category therefore means that SecurePR did not identify a mapped automated finding for that category. It does not mean that the entire category has been proven secure.

8 Controlled Experiments

The system was tested using controlled pull requests to verify the complete security-gate lifecycle.

8.1 Experiment 1: Clean Pull Request

A harmless change was submitted in a pull request against the main branch.

The expected behavior was:

Clean Change → PASS

The SecurePR Security Gate completed successfully.

This demonstrated that normal development changes can pass through the security gate when the configured controls do not identify a blocking condition.

8.2 Experiment 2: Hardcoded Credential

A synthetic credential was intentionally introduced for testing:

```
password = "demo_password"
```

The value was not a real password or production credential.

The expected behavior was:

Hardcoded Credential → Semgrep Finding → BLOCK

The resulting workflow blocked the pull request. The security summary identified the Semgrep finding and reported the applicable OWASP categories.

8.3 Experiment 3: Remediation

The synthetic credential was then removed from the same pull request.

Because SecurePR is triggered again when a pull request is updated, the workflow reran automatically.

The expected behavior was:

Credential Removed → Workflow Reruns → PASS

The rerun successfully passed.

This demonstrates the intended development lifecycle:

Submit → Detect → Block → Fix → Rerun → Pass

9 Visual Evidence

Screenshots are included because the primary purpose of SecurePR is to provide an observable CI/CD security process. The screenshots document the actual workflow behavior rather than simply describing it.

9.1 CI PASS Result

Figure 2 shows the GitHub Actions result for a clean pull request.

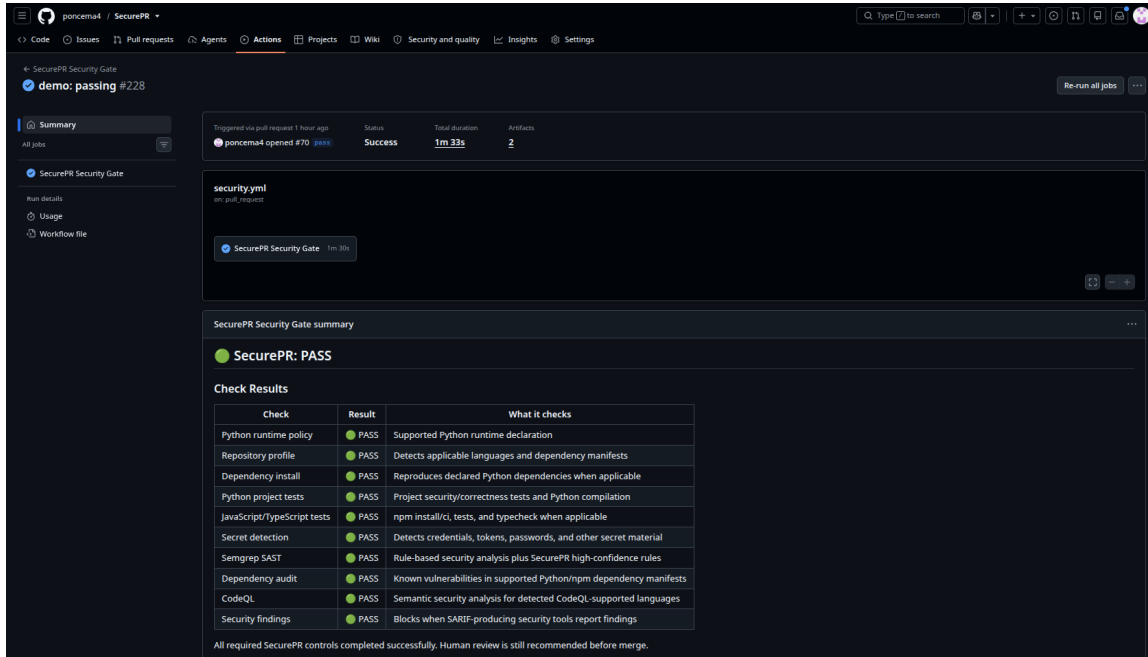


Figure 2: Successful SecurePR CI workflow result for a clean pull request.

9.2 PR BLOCK Result

Figure 3 shows the GitHub Actions result after introducing the synthetic hardcoded credential.

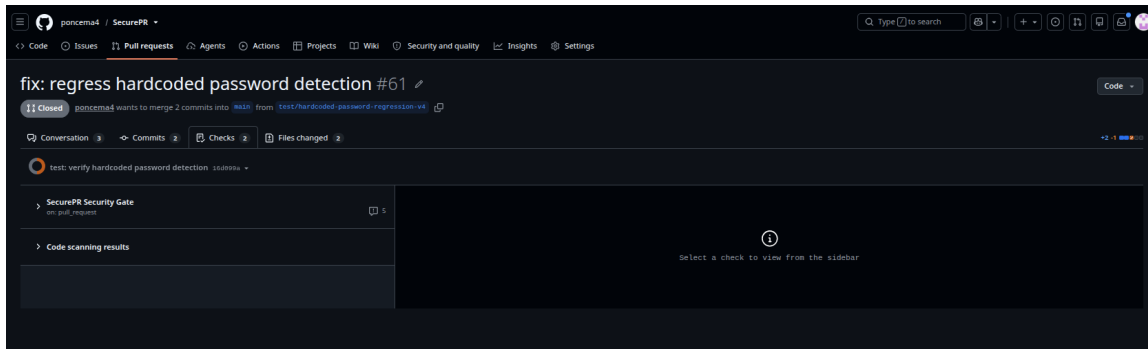


Figure 3: SecurePR BLOCK result after introducing a synthetic hardcoded credential. A04 and A07 are blocked because the detected finding is explicitly mapped to those categories.

9.3 PR PASS Result

Figure 4 shows the same pull request after the hardcoded credential was removed and the workflow reran.

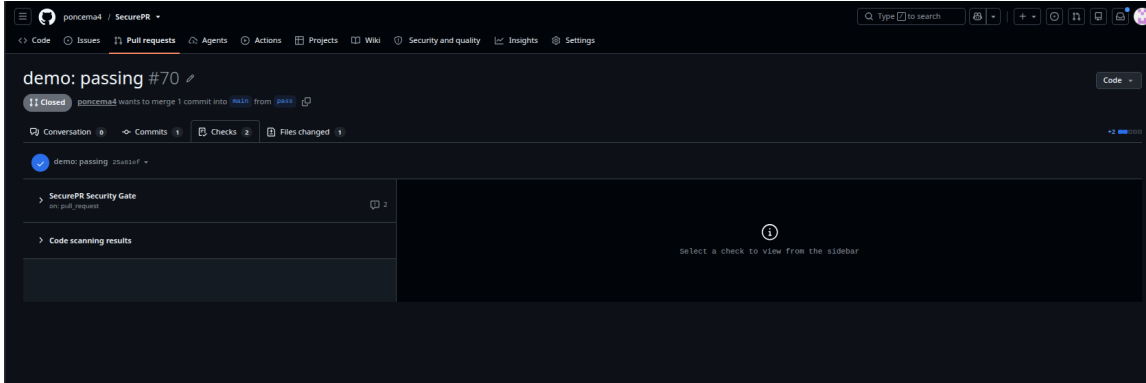


Figure 4: Successful SecurePR PASS result after the hardcoded credential was removed from the same pull request.

10 Reusable Workflow

A major design goal of SecurePR is reuse.

GitHub supports reusable workflows that allow one workflow to be called by another repository's workflow [2]. This avoids requiring developers to duplicate the complete security implementation in every repository.

A simplified consumer workflow has the following structure:

```
name: SecurePR

on:
  pull_request:
    branches:
      - main

jobs:
  securepr:
    uses: poncema4/SecurePR/.github/workflows/reusable-security.yml@REVIEWED_SHA
```

GitHub documents that reusable workflows are called directly at the job level using the **uses** keyword. GitHub also identifies a commit SHA as the safest reference for stability and security when consuming a reusable workflow [2].

This design separates the security harness from the application repository.

The SecurePR repository can therefore maintain the security workflow while another project calls the reviewed version.

Cross-repository validation is considered future work for this report unless the consumer repository has been independently tested.

11 Accuracy Evaluation

SecurePR includes a reviewed benchmark containing 37 labeled test cases.

The benchmark produced the following results:

Metric	Result
True Positives	15
False Positives	0
True Negatives	16
False Negatives	6
Total Cases	37
Accuracy	83.78%
Precision	100.00%
Recall	71.43%
F1 Score	83.33%

Table 3: SecurePR reviewed benchmark results.

Accuracy was calculated as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{15 + 16}{37} = 83.78\%$$

Precision was calculated as:

$$Precision = \frac{TP}{TP + FP} = \frac{15}{15 + 0} = 100\%$$

Recall was calculated as:

$$Recall = \frac{TP}{TP + FN} = \frac{15}{15 + 6} = 71.43\%$$

The F1 score was:

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall} = 83.33\%$$

The results demonstrate that the reviewed test corpus contained no false positives, while six cases were not detected.

The benchmark should not be interpreted as “SecurePR is 83.78% accurate in the real world.” It is a measurement of the implemented system against the project’s reviewed 37-case benchmark.

The false negatives are particularly important because they demonstrate that the system does not detect every security condition. This reinforces the need for human review and additional security testing.

12 Verification and Results

The project was verified through both automated benchmark testing and live pull-request experiments.

The live experiments demonstrated the intended security-gate lifecycle. A clean change passed the gate, an intentionally introduced hardcoded credential caused a BLOCK, and removing the credential caused the same pull request to pass after the workflow reran.

The OWASP reporting implementation was also tested independently. A hardcoded credential finding correctly mapped to A04 and A07 rather than causing all OWASP categories to fail.

The benchmark results provide a second form of verification by measuring the system against known expected outcomes.

Together, these tests demonstrate that SecurePR functions as a working pull request security gate rather than only as a conceptual architecture.

13 Limitations

SecurePR has several limitations.

First, the benchmark contains six false negatives. The system therefore cannot be considered a complete vulnerability detector.

Second, the current custom Semgrep rules cover a limited number of security conditions. Additional security rules would be required for broader vulnerability coverage.

Third, security-tool coverage depends on the capabilities of the underlying tools. CodeQL, for example, supports a defined set of programming languages and frameworks [3].

Fourth, OWASP mapping is limited to findings for which SecurePR has an explicit mapping. A passing OWASP category does not prove that the category is completely secure.

Fifth, dependency auditing depends on the available vulnerability databases and ecosystem support.

Finally, automated security analysis cannot replace human review, threat modeling, architecture analysis, or manual security testing.

These limitations are important because the purpose of SecurePR is to improve the development security process, not to claim that a successful automated run proves software is vulnerability-free.

14 Future Work

Several improvements could be made in future versions of SecurePR.

The first would be expanding the custom security-rule library to cover additional vulnerability classes such as SQL injection, command injection, path traversal, insecure deserialization, and weak cryptographic configurations.

The benchmark could also be expanded with additional languages, vulnerability classes, and negative examples. A larger independent evaluation set would provide a stronger measurement of detection behavior.

Another area of future work is cross-repository validation. SecurePR's reusable workflow should be tested against multiple independent repositories with different programming languages and dependency structures.

The project could also add richer developer remediation information, such as recommended fixes,

CWE identifiers, severity information, and links to internal security guidance.

Finally, additional supply-chain controls could be integrated in future versions. OWASP identifies source control, dependencies, build systems, and CI/CD infrastructure as important parts of software supply-chain security [7]. SecurePR could therefore eventually expand beyond pull-request scanning into stronger build and artifact integrity controls.

15 Conclusion

SecurePR demonstrates a practical approach to integrating security into the pull-request development process.

The project does not attempt to replace established security-analysis tools. Instead, it provides a reusable orchestration and policy layer around project tests, dependency auditing, Gitleaks, Semgrep, CodeQL, and SARIF-based security evidence.

The controlled experiments demonstrated the intended behavior of the security gate. A clean pull request passed, a synthetic hardcoded credential caused the pull request to be blocked, and removing the credential allowed the same pull request to pass after the workflow reran.

The reviewed 37-case benchmark produced 83.78% accuracy, 100% precision, 71.43% recall, and an F1 score of 83.33%. These measurements demonstrate the behavior of the implemented system against the project’s reviewed test corpus while also showing that additional detection coverage is needed.

The project also demonstrates why security reporting must be precise. The corrected OWASP implementation maps actual findings to applicable categories instead of treating a single scanner failure as evidence that unrelated OWASP categories are insecure.

Overall, SecurePR provides a practical example of DevSecOps-oriented security automation. Its main contribution is a reusable mechanism for coordinating security checks, enforcing a consistent pull-request policy, preserving evidence, and giving developers a clear PASS or BLOCK result while recognizing the continued importance of human security review.

References

- [1] GitHub. *Workflows*. GitHub Actions Documentation. 2026. <https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows>
- [2] GitHub. *Reuse workflows*. GitHub Actions Documentation. 2026. <https://docs.github.com/en/actions/how-to/reuse-automations/reuse-workflows>
- [3] GitHub. *Code scanning with CodeQL*. GitHub Code Security Documentation. 2026. <https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-code-scanning>
- [4] Gitleaks. *Gitleaks: Protect and find secrets*. 2026. <https://gitleaks.org/>
- [5] Souppaya, M., Scarfone, K., & Dodson, D. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication 800-218. National Institute of Standards and Technology, 2022. <https://doi.org/10.6028/NIST.SP.800-218>
- [6] OWASP Foundation. *OWASP Top 10:2025*. Open Worldwide Application Security Project, 2025. <https://top10.owasp.org/2025/>
- [7] OWASP Foundation. *Software Supply Chain Security Cheat Sheet*. OWASP Cheat Sheet Series. 2026. https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html
- [8] Semgrep. *Semgrep Documentation*. 2026. <https://semgrep.dev/docs/>